KERALA POLYTECHNIC HANDBOOK • PROGRAM ELECTIVE

Verilog HDL and Programmable Logic Devices

A full-screen syllabus-based handbook for Course Code **6041B**. This is not a simple notebook. It is a big handbook covering Verilog program structure, modelling styles, simulation thinking, common HDL mistakes, CPLD/FPGA architecture, solved examples, exam-style answers and Malayalam support notes.

Start learning | Model paper

PROGRAM
Diploma in Electronics Engineering

SEMESTER
6

CREDITS
4

PERIODS
60 / semester

□ subject-□□□□ main idea: software □□□□ code □□□□□□□□□□□□□□□□□□□□ output □□□ hardware circuit □□□. Verilog □□□□□□□□□□□□□□ syntax □□□□□□ □□□□□□□□□ □□□□; □□□ statement hardware □□□ □□□□□ □□□□□ □□□□□□□□□□□□□□.

SYLLABUS STUDY MAP

Official structure converted into handbook plan

The syllabus has 60 periods, 4 credits and four course outcomes. This page follows that structure and expands each outcome into study notes, examples and exam practice.

Course objectives

- ✓ Understand basics of Hardware Description Language.
- ✓ Use Verilog HDL for digital circuit design.
- ✓ Understand programmable logic devices.
- ✓ Connect Verilog modelling with CPLD/FPGA implementation.

Prerequisites

- Electronic components and circuits.
- Combinational and sequential digital circuits.
- Basic programming concepts.

> Digital Electronics strong □□□□□□□□□□□ Verilog difficult □□□□□□□□□□. Boolean algebra, flip-flop, counter, multiplexer □□□□□ revise □□□□□□□.

CO	Outcome	Hours	Handbook focus
CO 1	Outline HDL basics and program structure.	14	CAD evolution, HDL flow, hierarchy, design block, stimulus block, data types, system tasks, modules and ports.
CO 2	Use gate level and dataflow modelling.	15	Gate primitives, delays, continuous assignment, operators, full adder, MUX, DFF, ripple counter.
CO 3	Use behavioural modelling.	15	initial, always, blocking/non-blocking, if/case/loops, combinational and sequential examples.
CO 4	Outline CPLD and FPGA architecture.	14	PAL, PLA, CPLD, FPGA, CLB, applications and comparison.
Tests	Series Test I and II	2	Model paper and answer key included.

FOUNDATION

How digital design changes after HDL

Traditional design path

1. Understand requirement.
2. Prepare truth table or state diagram.
3. Simplify Boolean expression.
4. Select ICs or gates.
5. Wire and test manually.

HDL design path

1. Write Verilog module.
2. Write testbench stimulus.
3. Simulate waveform.
4. Synthesize to gates/LUTs/flip-flops.
5. Implement in CPLD/FPGA and test hardware.

Exam point: HDL is not simply a programming language. Verilog describes hardware structure, data movement and timing. Many assignments may represent parallel hardware.

CO1 • 14 HOURS

Module 1: HDL Basics and Verilog Program Structure

CAD evolution, HDL need, hierarchy, design/stimulus blocks, data types, system tasks, compiler directives, modules and ports.

Evolution of CAD

CAD moved digital design from hand-drawn gate diagrams to schematic capture, simulation, synthesis, place-and-route and timing analysis.

Why HDL?

HDL gives a text-based, reusable and simulatable representation of hardware. The same design can be checked with a testbench before hardware programming.

Top-down design

Start with the whole system, divide it into modules, then write each sub-module.

Bottom-up design

Design and test small blocks first, then connect them to form larger blocks.

Verilog module structure

```
module module_name (port_list);
    input  a, b;
    output y;
    // internal logic
endmodule
```

Remember: module definition is the blueprint. Module instance is a copy used inside another module.

Design block vs stimulus block

The design block is the actual circuit. The stimulus block or testbench applies inputs and checks outputs.

Design block	Stimulus block
Synthesizable hardware	Verification only
Mapped to gates/LUTs	Not programmed into FPGA

Lexical conventions

- Whitespace separates tokens.
- Comments use `//` or `/* ... */`.
- Numbers use size'base value, e.g. `4'b1010`, `8'hA5`.
- Keywords include `module`, `input`, `output`, `wire`, `reg`, `always`, `assign`, `endmodule`.

Data types

wire represents a net driven by continuous assignment or gate output. **reg** stores a value assigned inside procedural blocks.

System tasks

- `$display` prints once.
- `$monitor` prints when a monitored signal changes.
- `$time` gives simulation time.
- `$finish` ends simulation.

Compiler directives

Directives begin with backtick. Common examples are `` `timescale ``, `` `define `` and `` `include ``.

CO2 • 15 HOURS

Module 2: Gate Level and Dataflow Modelling

Gate primitives, instantiation, delays, continuous assignment, operators and simple designs.

Gate level modelling

Uses predefined primitives such as `and`, `or`, `not`, `nand`, `nor`, `xor` and `xnor`.

```
module and_gate_gatelevel(input wire a,b, output wire y);
    and G1 (y, a, b);
endmodule
```

Gate instantiation rule

> gate_type instance_name (output, input1, input2, ...);

For primitive gates, output is written first. This is a common exam mistake.

Full adder using dataflow

```
module full_adder_dataflow(input wire a,b,cin, output wire sum,cout);
    assign sum  = a ^ b ^ cin;
    assign cout = (a & b) | (b & cin) | (a & cin);
endmodule
```

Continuous assignment

Continuous assignment continuously drives a wire. Whenever RHS input changes, LHS updates.

> assign y = a & b;

Implicit continuous assignment: `wire y = a & b;`

Delays

```
and #(3) G1 (y, a, b);       // single delay
and #(2, 4) G2 (y, a, b);    // rise/fall delay
and #(2:3:5) G3 (y, a, b);   // min:typ:max
```

Synthesis warning

Many delay statements are simulation concepts. FPGA synthesis usually ignores arbitrary # delays.

> #10 delay □□□□□□□□□ FPGA-□□ 10 ns delay gate □□□□□□□□□□□□□ □□□□□□□□.
> □□□ simulation timing □□□□□□ □□□.

Operators

Type	Operators	Use
Bitwise	&, |, ^, ~	Operate bit by bit
Logical	&&, ||, !	True/false decision
Arithmetic	+, -, *, /	Add/subtract etc.
Conditional	? :	2-way selection
Concatenation	{ }	Join bits

CO3 • 15 HOURS

Module 3: Behavioural Modelling

initial, always, assignments, delays, event control, if/case/loops and design examples.

initial block

An `initial` block starts once at simulation time zero and is mainly used in testbenches.

```
initial begin
    a = 0; b = 0;
    #10 a = 1;
end
```

always block

```
always @(*) begin
    y = a & b;
end

always @(posedge clk) begin
    q <= d;
end
```

Blocking vs non-blocking

Assignment	Symbol	Typical use

Blocking	=	Combinational procedural logic
Non-blocking	<=	Clocked sequential logic

4:1 MUX behavioural

```
module mux4_behavioural(input wire [3:0] d,input wire [1:0] sel,output
reg y);
    always @(*) begin
        case (sel)
            2'b00: y = d[0];
            2'b01: y = d[1];
            2'b10: y = d[2];
            2'b11: y = d[3];
            default: y = 1'b0;
        endcase
    end
endmodule
```

D flip-flop

```
module d_ff(input wire clk,rst,d, output reg q);
    always @(posedge clk or posedge rst) begin
        if (rst) q <= 1'b0;
        else     q <= d;
    end
endmodule
```

4-bit counter

```
module counter4(input wire clk,rst, output reg [3:0] q);
    always @(posedge clk or posedge rst) begin
        if (rst) q <= 4'b0000;
        else     q <= q + 1'b1;
    end
endmodule
```

CO4 • 14 HOURS

Module 4: PAL, PLA, CPLD and FPGA Architecture

PAL and PLA

PLA has programmable AND array and programmable OR array. **PAL** has programmable AND array and fixed OR array.

Feature	PAL	PLA
AND plane	Programmable	Programmable
OR plane	Fixed	Programmable
Flexibility	Medium	High

CPLD architecture

A CPLD contains multiple logic blocks or macrocells connected using a programmable interconnect matrix. It is good for control logic, glue logic, address decoding and medium-complexity designs.

FPGA architecture

An FPGA contains configurable logic blocks, interconnect resources, I/O blocks and memory/DSP resources. It is suitable for large parallel digital systems.

Configurable Logic Block

A CLB usually contains LUTs, flip-flops, multiplexers and carry logic.

> A 4-input LUT can implement any Boolean function of 4 variables.

> CLB □□□□□□ FPGA-□□□□ repeated logic cell □□□. Verilog code synthesize □□□□□□□ LUT, flip-flop, routing resources □□□ map □□□□□□□.

CPLD vs FPGA

Point	CPLD	FPGA

Best for	Control and glue logic	Large parallel digital systems
Capacity	Low to medium	Medium to very high
Timing	More predictable	Depends on routing/place-and-route
Configuration	Often non-volatile	Often SRAM based

PRACTICAL WORKFLOW

How to write and verify Verilog

1. Write problem statement.
2. Draw truth table, block diagram or state diagram.
3. Choose gate, dataflow or behavioural style.
4. Write module with correct port directions.
5. Write separate testbench.
6. Run simulation and check waveform.
7. Correct syntax and logic errors.
8. Synthesize only after simulation is correct.

```
module tb_full_adder;
    reg a,b,cin;
    wire sum,cout;
    full_adder_dataflow dut(.a(a),.b(b),.cin(cin),.sum(sum),.cout(cout));
    initial begin
        $monitor("%b %b %b -> %b %b",a,b,cin,sum,cout);
        a=0;b=0;cin=0; #10 cin=1; #10 b=1; #10 a=1; #10 $finish;
    end
endmodule
```

Common mistakes: missing semicolon, wrong port names, assigning wire inside always, incomplete case statement causing latch, using testbench delay as real hardware delay, mixing blocking and non-blocking without timing understanding.

SOLVED EXAMPLES

Design examples for practice

2:1 MUX dataflow

```
module mux2_df(input wire a,b,sel, output wire y);
    assign y = sel ? b : a;
endmodule
```

Half adder gate level

```
module half_adder_gate(input wire a,b, output wire sum,carry);
    xor X1(sum, a, b);
    and A1(carry, a, b);
endmodule
```

2 to 4 decoder

```
module decoder2to4(input wire [1:0] a, output reg [3:0] y);
    always @(*) begin
        case(a)
            2'b00: y = 4'b0001;
            2'b01: y = 4'b0010;
            2'b10: y = 4'b0100;
            2'b11: y = 4'b1000;
        endcase
    end
endmodule
```

Ripple carry adder idea

A ripple carry adder is built by cascading full adders. Carry-out of one stage becomes carry-in of next stage.

> Worst-case carry delay ≈ n × full-adder carry delay

QUICK REVISION

High-value facts before exam

M1

- HDL describes hardware.
- Design and stimulus blocks are different.
- Module is basic design unit.

M2

- Gate primitive output first.
- Dataflow uses assign.
- Delays are mainly simulation.

M3

- initial runs once.
- always repeats.
- = for combinational, <= for sequential.

M4

- PAL fixed OR.
- PLA programmable OR.
- FPGA uses CLB/LUT.

Modelling styles

Style	Best use	Main keyword	Example

Gate level	Small gate diagram circuits	and, or, not	Half adder
Dataflow	Boolean equations	assign	Full adder, MUX
Behavioural	Algorithm-like hardware	always, initial	Counter, FSM

QUESTION BANK

Expected questions module-wise

Short answer

Q1 Define HDL and list two advantages.

Q2 Differentiate module and module instance.

Q3 What is a stimulus block?

Q4 Write syntax of a Verilog module.

Q5 List four Verilog data types.

Q6 What are gate primitives?

Q7 Define continuous assignment.

Q8 What is gate delay?

Q9 Differentiate blocking and non-blocking assignment.

Q10 What is a configurable logic block?

Long answer/design

Q1 Explain HDL based design flow with diagram.

Q2 Explain hierarchical modelling.

Q3 Write Verilog for full adder.

Q4 Design 4:1 MUX using behavioural modelling.

Q5 Explain always and initial blocks.

Q6 Write Verilog for a 4-bit counter.

Q7 Compare PAL and PLA.

Q8 Explain CPLD architecture.

Q9 Explain FPGA architecture and CLB.

Q10 Compare FPGA and CPLD.

MODEL PAPER

Practice question paper with answer key

Course Code: 6041B
Course Title: Verilog HDL and Programmable Logic Devices
Time: 3 Hours Maximum Marks: 100

PART A - Answer any ten questions. Each carries 3 marks.
1. Define Hardware Description Language.
2. State any three advantages of Verilog HDL.
3. What is the difference between top-down and bottom-up design?
4. Define module and port in Verilog.
5. What is the use of system task $monitor?
6. List four gate primitives in Verilog.
7. What is continuous assignment?
8. Mention two types of delay statements in dataflow modelling.
9. What is the purpose of always block?
10. Distinguish blocking and non-blocking assignment.
11. Define PAL and PLA.
12. What is a CLB in FPGA?

PART B - Answer any five questions. Each carries 8 marks.
13. Explain typical HDL based design flow.
14. Explain the structure of a Verilog module with example.
15. Write Verilog code for full adder using gate level or dataflow modelling.
16. Explain dataflow operators used in Verilog with examples.
17. Explain initial and always procedures in behavioural modelling.
18. Write Verilog code for 4:1 multiplexer using behavioural modelling.
19. Compare PAL and PLA.
20. Compare CPLD and FPGA.

PART C - Answer any three questions. Each carries 10 marks.

21. Explain hierarchical modelling concepts and components of simulation.
22. Explain gate delays and continuous assignment delays in Verilog.
23. Develop Verilog program for D flip-flop and 4-bit counter.
24. Explain CPLD architecture with block schematic and applications.
25. Explain FPGA architecture, configurable logic blocks and applications.

ANSWER KEY

Concise answers

Part A answer points

1. HDL describes digital hardware structure and behaviour.
2. Advantages: simulation before hardware, reusable modules, hierarchy, synthesis to FPGA/CPLD.
3. Top-down starts from system; bottom-up builds small blocks and integrates.
4. Module is design unit; port is input/output/inout connection.
5. `$monitor` displays values when monitored variables change.
6. and, or, not, nand, nor, xor, xnor.
7. Continuous assignment drives a net continuously using `assign`.
8. Regular assignment delay, implicit assignment delay, net declaration delay.
9. `always` describes repeated procedural block triggered by sensitivity list.
10. Blocking = executes sequentially; non-blocking <= schedules update for clocked logic.
11. PAL: programmable AND/fixed OR. PLA: programmable AND/programmable OR.
12. CLB is FPGA logic block containing LUTs, flip-flops and selection logic.

Long answer guidance

For 8-mark and 10-mark answers, write definition, neat diagram or code, explanation, advantages/applications and comparison. For design questions, include module declaration, port directions, core logic and verification note. For CPLD/FPGA, draw I/O blocks, programmable interconnect, macrocells or CLBs, configuration memory and explain signal flow.

REFERENCES

Books and online resources from syllabus

Text books

- Samir Palnitkar, Verilog HDL, Pearson Education.

- M. Morris Mano, Digital Design, PHI.

Reference books

- Morris Mano and Michael D. Ciletti, Digital Design with Verilog HDL, VHDL and SystemVerilog.
- Wayne Wolf, FPGA Based System Design.
- T. R. Padmanabhan and B. Bala Tripura Sundari, Design Through Verilog HDL.
- Pong P. Chu, FPGA Prototyping by Verilog Examples.
- Nazeih M. Botros, HDL Programming.
- Ian Grout, Digital Systems Design with FPGAs and CPLDs.